

WORKSHEET MAHASISWA

Modul 2 — Vanilla JavaScript dan Interaktivitas Web

Dasar Java Script · DOM dan Event · Validasi Form · Asynchronous · Debugging

Mata Kuliah Proyek 3 — Proyek Pengembangan Perangkat Lunak Berbasis Web · D3 Teknik Informatika

Identitas	Isian
Nama	Immanoel
NIM	251511042
Kelas	2B – D3
Tanggal Mulai	14, September
Tanggal Pengumpulan	
URL Repository	https://github.com/tiranui/Proyek3-Minggu2-Immanoel

Petunjuk Penggunaan

- ☐ Isi worksheet bersamaan dengan pengerjaan modul, bukan setelah seluruh tugas selesai.
- ☐ Pada eksperimen, tulis prediksi sebelum menjalankan kode, lalu catat hasil aktual dan bukti.
- ☐ Gunakan Console, Elements, Network, dan tampilan halaman sebagai sumber bukti sesuai aktivitas.
- ☐ Gunakan hint secara bertahap; catat percobaan sebelum membuka hint berikutnya.
- ☐ Catat penggunaan AI atau sumber bantuan secara transparan dan lakukan verifikasi mandiri.
- ☐ Worksheet dikumpulkan dalam PDF bersama bukti yang disyaratkan pada tugas.

Alur Belajar Alur pembelajaran dilaksanakan melalui lima tahap, yaitu membuat prediksi, menjalankan percobaan, mengamati hasil, menjelaskan temuan, dan melakukan perbaikan. Bukti proses memiliki kedudukan yang sama pentingnya dengan hasil akhir.

A. Eksperimen

A1. Eksperimen Console dan Tipe Data

Isi kolom *Prediksi* sebelum membuka *Console*. Setelah menjalankan kode, catat hasil aktual dan alasan perbedaannya.

Ekspresi atau Pemeriksaan	Prediksi	Hasil Aktual	Tipe Data atau Makna	Penjelasan
typeof nama	Rani	string	string	Typeof merupakan operasi yang digunakan untuk mengetahui tipe data dari sebuah variable. Nama = rani saat typeof dijalankan menghasilkan tipe data dari variable Nama yang merupakan string
typeof nilaiTeks	80	string	string	Typeof merupakan operasi yang digunakan untuk mengetahui tipe data dari sebuah variable. NilaiTeks = '80' saat typeof dijalankan menghasilkan tipe data dari variable NilaiTeks yang merupakan string
typeof nilaiAngka	80	number	number	Typeof merupakan operasi yang digunakan untuk mengetahui tipe data dari sebuah variable. NilaiAngka = 80 saat typeof dijalankan menghasilkan tipe data dari variable NilaiAngka yang merupakan number
nilaiTeks + 5	805	805	String + number	NilaiTeks merupakan tipe data string, saat melakukan operasi aritmatika yang akan terjadi adalah penggabungan string + number yang menggabungkan keduanya menjadi string
Number(nilaiTeks) + 5	85	85	Number + number	NilaiTeks merupakan tipe data string yang isinya angka, saat digunakan operasi number yang terjadi adalah string berubah menjadi integer. Saat melakukan operasi aritmatika yang terjadi adalah number + number yang menghasilkan number

Ekspresi atau Pemeriksaan	Prediksi	Hasil Aktual	Tipe Data atau Makna	Penjelasan
nilaiTeks == nilaiAngka	True	True	boolean	Sama dengan dua merupakan perbandingan nilai dari variable tanpa membandingkan tipe data
nilaiTeks === nilaiAngka	True	False	boolean	Sama dengan tiga merupakan perbandingan nilai dari variable membandingkan tipe data
Number(teks bukan angka)	Error	NaN	number	Hasilnya NaN karena Number(nama) merupakan string yang nilainya bukan angka

Apa bukti bahwa angka 80 dan teks "80" berbeda?

Angka 80 merupakan tipe data number atau integer dan teks "80" merupakan tipe data string

Mengapa operator + dapat menghasilkan hasil yang berbeda?

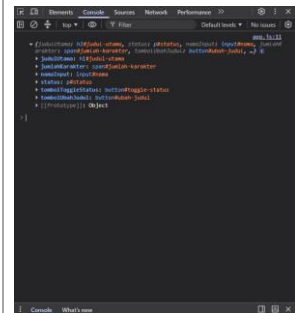
Karena nilai teks + 5 merupakan tipe data string + number yang terjadi penggabungan yang akan menghasilkan tipe data string, tapi nilai angka + 5 merupakan tipe data number + number yang menghasilkan tipe data number

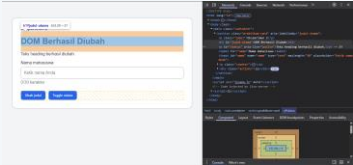
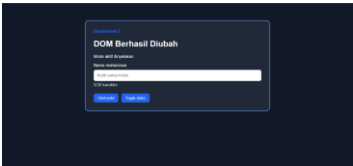
Mengapa perbandingan ketat === menjadi pilihan awal yang lebih aman?

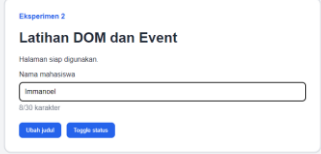
Karena sama dengan tiga merupakan perbandingan variable dengan membandingkan nilai dan tipe datanya

A2. Eksperimen DOM dan Event

Alur yang diamati Alur yang diamati dimulai ketika pengguna melakukan suatu aksi. Browser kemudian membangkitkan event, fungsi penanganan dijalankan, elemen DOM atau class diperbarui, dan browser merender perubahan pada tampilan.

Langkah	Prediksi sebelum diuji	Hasil aktual	Bukti dan Penjelasan
1 · Memilih elemen	Element tidak ada yang null	Tidak ada element yang null	

Langkah	Prediksi sebelum diuji	Hasil aktual	Bukti dan Penjelasan
2 · Mengubah heading	Heading akan berubah menjadi Latihan DOM dan Event	Setelah tombol ubah judul di klik yang terjadi adalah heading akan berubah menjadi Latihan Latihan DOM dan Event	
3 · Mengubah class dan atribut aria-pressed	Saat button toggle status di click lalu akan mengubah tag toggle dari false menjadi true, mengubah tulisan status ? 'Mode aktif dinyalakan.' : 'Mode aktif dimatikan.'; Warna pun akan berubah menjadi border color biru dan background color hitam sesuai is-active pada file css	Hasil aktual dari fitur mengubah class dan atribut aria-pressed adalah ketika tombol "Toggle status" diklik, class is-active pada elemen <body> akan ditambahkan sehingga tampilan halaman berubah menjadi mode gelap, kemudian atribut aria-pressed pada tombol berubah dari false menjadi true dan teks status berubah menjadi "Mode aktif dinyalakan." Ketika tombol diklik kembali, class is-active dihapus, aria-pressed kembali menjadi false, tampilan kembali seperti semula, dan teks status berubah menjadi "Mode aktif dimatikan."	

Langkah	Prediksi sebelum diuji	Hasil aktual	Bukti dan Penjelasan
4 · Event input dan penghitung karakter	Menghitung banyak huruf yang ada pada box “Ketik nama Anda” sesuai input yang kita berikan, bisa juga menghitung huruf jika kita input menempelkan teks tapi terbatas 30 huruf	Hasil aktual dari fitur penghitung jumlah karakter adalah ketika pengguna mengetik nama pada kolom “Nama mahasiswa”, jumlah karakter akan langsung dihitung dan ditampilkan pada bagian penghitung. Sebagai contoh, ketika pengguna memasukkan “Immanuel”, maka tampilan berubah dari “0/30 karakter” menjadi “8/30 karakter”. Jumlah tersebut juga akan berubah secara otomatis setiap kali pengguna menambah atau menghapus karakter.	 The screenshot shows a web form titled "Latihan DOM dan Event". It has a text input field labeled "Nama mahasiswa" containing the text "Immanuel". Below the input field, the character count is displayed as "8/30 karakter". There are two buttons at the bottom: "Ukuri Jaki" and "Ukuri Saku".

Langkah	Prediksi sebelum diuji	Hasil aktual	Bukti dan Penjelasan
5 · Selector tidak cocok dan menghasilkan nilai null	Akan terjadi error pada bagian ubah judul karena ID status berubah berbeda dengan bagian selector js	Setelah setiap baris <code>status.textContent = ...</code> pada event handler diganti dengan pemanggilan fungsi <code>ubahStatus()</code> , kerusakan selector kembali diuji dengan mengubah <code>id="status"</code> menjadi <code>id="status-pesan"</code> . Saat tombol Ubah judul diklik, tidak lagi muncul error <code>TypeError</code> seperti sebelumnya, tetapi muncul pesan peringatan pada Console yaitu “Elemen #status tidak ditemukan.” Hal ini terjadi karena pemeriksaan <code>if (!status)</code> mendeteksi bahwa elemen dengan selector <code>#status</code> tidak ditemukan dan menghentikan fungsi dengan <code>return</code> . Setelah id dipulihkan menjadi <code>status</code> , halaman kembali berjalan normal dan pesan status dapat ditampilkan melalui fungsi <code>ubahStatus()</code> .	

Tuliskan pesan error ketika selector tidak cocok, meliputi nama file, nomor baris, dan inti pesan:

```
app.js:22 Uncaught ReferenceError: ubahStatus is not defined at HTMLButtonElement.<anonymous>
```

Jelaskan perbedaan bukti yang diperoleh sebelum dan sesudah pemeriksaan nilai null melalui function ubahStatus.

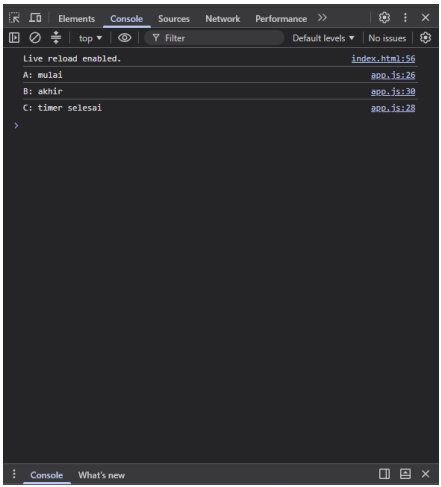
Sebelum menggunakan pemeriksaan nilai null dengan function ubahStatus(), Ketika selector status tidak cocok, console akan menampilkan error uncaught TypeError. Setelah menggunakan function ubahStatus() program menampilkan “Elemen #status tidak ditemukan.” Jadi, perbedaannya adalah sebelum pemeriksaan null terjadi error yang menghentikan proses, sedangkan sesudah pemeriksaan null kesalahan dapat ditangani dengan pesan peringatan yang lebih jelas.

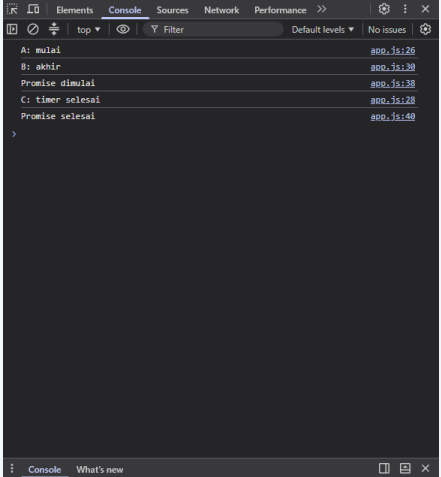
Jelaskan pembagian peran JavaScript dan CSS ketika class is-active diubah melalui mekanisme toggle.

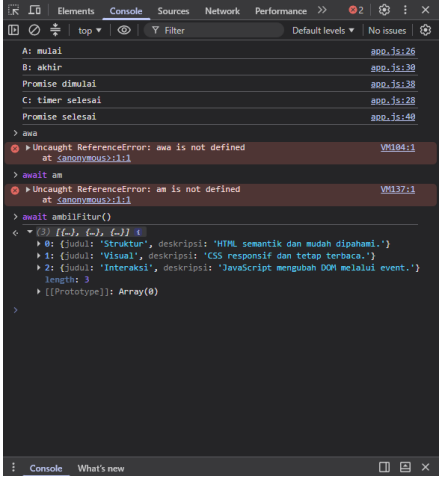
Saat class is-active berubah peran dari JavaScript adalah mengubah class pada element body, jadi saat tombol diklik akan merubah aria-pressed apakah menjadi true atau false. File css berguna untuk mengubah tampilan dari background dan border dari website, dengan selector body,is-active.

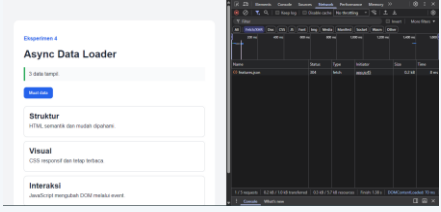
A3. Eksperimen Promise, fetch, dan State UI

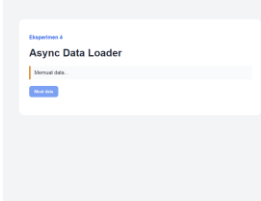
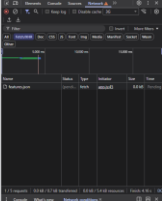
Jalankan proyek melalui server pengembangan lokal. Gunakan tampilan halaman, Console, dan Network sebagai bukti pengujian.

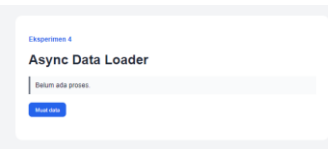
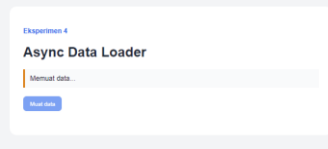
Skenario atau Langkah	Prediksi	Hasil Aktual	Bukti dari Console, Network, atau Antarmuka
Urutan A, B, C pada setTimeout(0)	Di console akan keluar output A: mulai, B: akhir, C: timer selesai	Output menghasilkan A: mulai, B: akhir, C:timer selesai secara berurutan	

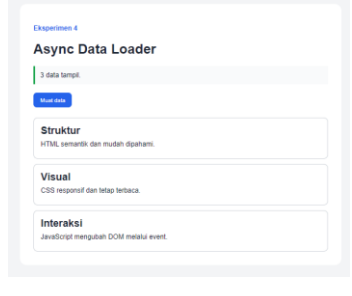
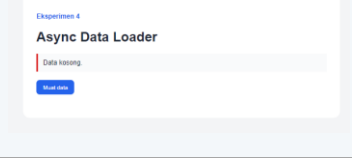
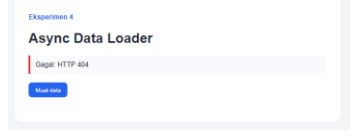
Skenario atau Langkah	Prediksi	Hasil Aktual	Bukti dari Console, Network, atau Antarmuka
Data JSON valid	Di console akan menghasilkan promise selesai setelah 800ms website di reload	Setelah function <code>tunggu()</code> dan <code>ujiTunggu()</code> dijalankan, pada Console muncul pesan "Promise dimulai", kemudian terdapat jeda sekitar 800 ms sebelum muncul pesan "Promise selesai". Pada awal proses, Promise berada dalam state pending karena masih menunggu <code>setTimeout</code> selesai. Setelah waktu 800 ms berakhir, fungsi <code>resolve</code> dijalankan sehingga state Promise berubah menjadi fulfilled. Setelah itu, <code>await</code> selesai menunggu dan program melanjutkan ke perintah <code>console.log('Promise selesai')</code>	 <pre> A: mulai 800.js:26 B: akhir 800.js:30 Promise dimulai 800.js:38 C: timer selesai 800.js:28 Promise selesai 800.js:40 </pre>

Skenario atau Langkah	Prediksi	Hasil Aktual	Bukti dari Console, Network, atau Antarmuka
Array kosong []	Di console akan menghasilkan array dari file.json	Setelah panel Network dibuka dan await ambilFitur() dijalankan melalui Console, terlihat request features.json pada bagian Fetch/XHR. Request tersebut memiliki status HTTP 200 (OK) yang menunjukkan bahwa file berhasil ditemukan dan diambil. Pada bagian Response terlihat isi data JSON dari file features.json, sedangkan hasil await ambilFitur() pada Console berupa array yang berisi data fitur. Hal ini terjadi karena fetch() mengambil file terlebih dahulu, kemudian response.ok memeriksa keberhasilan request, dan response.json() mengubah data JSON menjadi array JavaScript yang dapat digunakan oleh program.	

Skenario atau Langkah	Prediksi	Hasil Aktual	Bukti dari Console, Network, atau Antarmuka
Path salah atau JSON rusak	Program akan menampilkan isi dari file.json dan akan dicek jika data bukan array maka akan output "Format data bukan array" jika data kosong maka output "empty, Data Kosong" jika berhasil output menghasilkan success data tampil	Setelah tombol "Muat Data" diklik, state pada halaman berubah menjadi loading dengan menampilkan tulisan "Memuat data..." dan tombol sementara dinonaktifkan selama proses pengambilan data berlangsung. Setelah proses fetch selesai dan data berhasil diterima dalam bentuk array serta tidak kosong, state kemudian berubah menjadi success dan menampilkan jumlah data yang berhasil dimuat, yaitu "5 data tampil." Dengan menggunakan Network throttling, proses loading dapat terlihat lebih lama sehingga urutan perubahan state dari loading → success dapat diamati dengan jelas.	 The screenshot shows a web application titled 'Async Data Loader'. It features a progress bar indicating '5 data tampil' (5 data displayed). Below the progress bar, there is a table with columns for 'id', 'nama', 'umur', 'alamat', and 'status'. The table contains 5 rows of data. The application also shows a 'Struktur' (Structure) section with a tree view and a 'Visual' section with a data visualization. The 'Interaksi' (Interaction) section shows a log of events.

Skenario atau Langkah	Prediksi	Hasil Aktual	Bukti dari Console, Network, atau Antarmuka
Klik berulang saat loading	saat mengaktifkan throttling dan mengklik tombol berkali kali	Pada pengujian klik berulang, Network throttling diaktifkan kemudian tombol “Muat Data” diklik beberapa kali secara cepat saat proses loading berlangsung. Hasil yang diamati menunjukkan bahwa setelah klik pertama, tombol menjadi disabled dan state berubah menjadi “Memuat data...”. Meskipun tombol diklik beberapa kali selama proses tersebut, tidak terjadi request ganda pada Network karena tombol tidak dapat digunakan kembali sampai proses pengambilan data selesai. Setelah data berhasil dimuat, state berubah menjadi success dan tombol kembali aktif.	 

State	Apa yang tampil	Kapan terjadi	Bukti
idle	Halaman siap, belum ada proses pemuat data.	Saat halaman pertama kali dibuka sebelum tombol Muat data diklik.	
loading	Muncul pesan "Memuat data...", daftar materi dikosongkan, dan tombol Muat data dinonaktifkan.	saat muatData() mulai menjalankan proses fetch.	

State	Apa yang tampil	Kapan terjadi	Bukti
success	Data materi berhasil ditampilkan sebagai kartu dan muncul pesan seperti "3 materi tampil."	Saat fetch berhasil dan JSON berisi data	
empty	Tidak ada kartu materi dan mu	Saat fetch berhasil tetapi isi JSON berupa array kosong [].	
error	Muncul pesan "Gagal memuat materi. Silakan coba lagi." dan tombol Coba lagi ditampilkan	aat fetch gagal, misalnya path salah/404, server mati, atau JSON rusak.	

Mengapa delay 0 pada setTimeout tetap tidak membuat callback berjalan sebelum kode sinkron selesai?

Karena setTimeout(..., 0) tidak berarti callback langsung dijalankan. Callback akan masuk ke task queue dan baru dijalankan setelah kode sinkron yang sedang berjalan selesai. Jadi JavaScript menyelesaikan kode sinkron terlebih dahulu, kemudian mengambil callback dari queue untuk dijalankan.

Apa perbedaan antara error HTTP atau kesalahan path dengan error parsing JSON berdasarkan bukti yang Anda peroleh?

Error HTTP/path terjadi ketika file tidak ditemukan atau server memberikan status error, misalnya 404 Not Found. Bukti dapat dilihat pada Network dengan status HTTP 404. Sedangkan error parsing JSON terjadi ketika file berhasil ditemukan dan respons diterima, tetapi isi JSON memiliki sintaks yang salah sehingga response.json() gagal membaca data. Bukti biasanya terlihat sebagai error parsing pada Console.

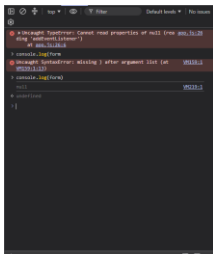
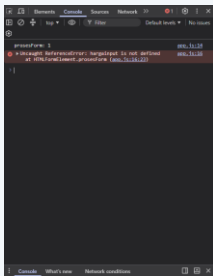
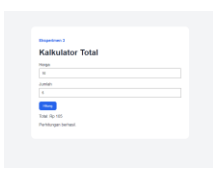
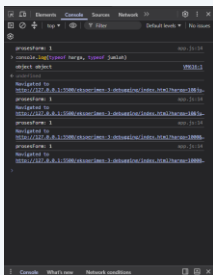
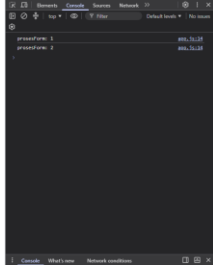
Mengapa finally cocok untuk mengaktifkan kembali tombol?

Karena finally selalu dijalankan setelah proses try selesai, baik proses tersebut berhasil maupun mengalami error tombol tidak akan tetap terkunci ketika fetch berhasil maupun ketika terjadi error.

A4. Eksperimen Diagnosis Error JavaScript

Metode Diagnosis Metode diagnosis dimulai dengan mencatat gejala dan hasil yang diharapkan, mereproduksi masalah, membaca pesan error pertama, memeriksa nilai atau selector, menyusun dugaan penyebab, melakukan satu perubahan, kemudian menguji ulang hasil perbaikan.

Kasus	Gejala dan Hasil yang Diharapkan	Pesan Error atau Nilai Aktual	Dugaan Penyebab	Perbaikan	Bukti Uji Ulang
-------	----------------------------------	-------------------------------	-----------------	-----------	-----------------

Kasus	Gejala dan Hasil yang Diharapkan	Pesan Error atau Nilai Aktual	Dugaan Penyebab	Perbaikan	Bukti Uji Ulang
A · Selector	Program seharusnya dapat menemukan elemen form dan input sehingga kalkulator dapat berjalan.	Jika selector salah, nilai variabel bisa null dan Console menampilkan error seperti Cannot read properties of null.	Selector pada querySelector() tidak sesuai dengan id pada HTML.	Menyamakan selector JavaScript dengan id elemen HTML, misalnya #form-hitung, #harga, dan #jumlah.	
B · Nama variabel	Variabel harus menunjuk ke elemen yang benar agar nilai harga, jumlah, hasil, dan pesan dapat digunakan.	Jika nama variabel salah atau tidak konsisten, dapat muncul ReferenceError: ... is not defined.	Penulisan nama variabel berbeda antara deklarasi dan penggunaannya.	Menyamakan nama variabel, misalnya hargaInput, jumlahInput, hasil, dan pesan.	
C · Tipe data dan operator	Harga dan jumlah harus berupa angka dan dikalikan untuk mendapatkan total.	Nilai aktual harus berupa hasil perkalian, misalnya harga 10000 dan jumlah 2 menghasilkan 20.000	Nilai dari input HTML awalnya berupa string atau operator yang digunakan tidak sesuai.	Menggunakan Number() untuk mengubah nilai input menjadi angka dan operator * untuk perkalian.	
D · Form reload	Setelah tombol Hitung ditekan, halaman seharusnya tidak melakukan reload sehingga hasil tetap tampil.	Jika tidak menggunakan preventDefault(), halaman dapat melakukan reload dan hasil perhitungan hilang.	Perilaku default form adalah melakukan submit/reload halaman.	Menambahkan event.preventDefault() pada prosesForm().	
E · Handler ganda	Fungsi prosesForm seharusnya hanya dijalankan satu kali untuk satu submit.	console.count('prosesForm') menunjukkan hitungan bertambah 2 kali untuk satu klik submit.	Event submit didaftarkan dua kali: satu langsung ke prosesForm dan satu lagi memanggil prosesForm.	Hapus salah satu event listener sehingga hanya ada satu handler submit.	

Kasus paling membantu pemahaman saya dan mengapa:

Handler ganda paling membantu pemahaman saya karena saya dapat mengetahui bahwa satu kali klik tombol dapat menjalankan fungsi dua kali jika event listener didaftarkan lebih dari satu kali. Dengan menggunakan `console.count('prosesForm')`, saya dapat melihat jumlah pemanggilan fungsi melalui Console sehingga lebih mudah memahami cara kerja event listener dan pentingnya menghindari handler ganda.

B. Pertanyaan Pemahaman

B1. Logika, Function, Array, dan Object

Kapan let lebih tepat digunakan daripada const?

Let digunakan saat nilai variabel akan berubah saat program berjalan, const digunakan saat nilai variabel tidak akan diganti

Mengapa nilai dari elemen input perlu dikonversi sebelum perhitungan?

Karena nilai input biasanya berupa string walaupun isinya angka. Agar dapat digunakan pengoperasian matematika diperlukan konvensi ke number

Mengapa kondisi kategori atau diskon dengan batas terbesar diperiksa lebih dahulu?

Agar kondisi yang lebih tinggi tidak terlebih dulu masuk ke kategori yang lebih rendah

Apa perbedaan parameter, argument, dan return?

Parameter merupakan variable yang ditulis setelah function

Argument merupakan nilai yang diberikan Ketika function dipanggil

Return merupakan nilai yang dikembalikan oleh function

Mengapa function logika sebaiknya dipisahkan dari function yang mengubah tampilan?

Agar program lebih struktur, mempermudah pencarian jika ada yang ingin di ubah.

Mengapa variabel lokal tidak dapat diakses dari luar function?

Karena variable local memiliki scope yang hanya bisa diakses di function tersebut

Mengapa kumpulan entitas dengan struktur sama cocok disimpan sebagai array of objects?

Karena array cocok untuk menyimpan data, sedangkan object cocok untuk mengelompokkan atribut yang memiliki Kumpulan entitas

Apa akibat mengakses indeks array yang tidak tersedia?

Javascript akan menghasilkan output pada console undefined

B2. DOM, Event, dan Validasi Form

Apa yang dikembalikan querySelector ketika selector tidak menemukan elemen?

`querySelector()` akan mengembalikan nilai null jika tidak menemukan elemen

Mengapa `classList` lebih tepat untuk perubahan visual daripada menulis style satu per satu di JavaScript?

Karena `classList` memungkinkan kita mengatur tampilan melalui CSS, sehingga JavaScript cukup mengubah class. CSS kemudian mengatur semua tampilan elemen tersebut

Apa hubungan antara `addEventListener`, event handler, dan perubahan DOM?

`ddEventListener()` digunakan untuk mendaftarkan event handler, yaitu function yang akan dijalankan ketika suatu event terjadi. Event handler tersebut kemudian dapat melakukan perubahan pada DOM

Mengapa atribut defer digunakan ketika app.js dipanggil dari head?

defer membuat browser mengunduh JavaScript tanpa menghentikan proses membaca HTML, tetapi menjalankan script setelah HTML selesai diproses.

Mengapa `textContent` digunakan untuk teks biasa atau data input pengguna?

Karena `textContent` memasukkan data sebagai teks biasa, bukan sebagai HTML yang harus diproses browser.

Apa fungsi `event.preventDefault()` pada submit form?

digunakan untuk mencegah perilaku default browser seperti melakukan reload atau navigasi halaman

Mengapa `trim` digunakan sebelum memvalidasi input teks?

`trim()` digunakan untuk menghapus spasi di awal dan akhir teks

Mengapa validasi sisi klien tetap tidak menggantikan validasi server?

Karena validasi JavaScript di browser dapat dilewati atau dimanipulasi oleh pengguna. Pengguna dapat menonaktifkan JavaScript, mengubah kode, atau mengirim request langsung ke server.

B3. Asynchronous JavaScript

Apa perbedaan state Promise pending, fulfilled, dan rejected?

Pending merupakan promise masih dalam pesan dan belum dapat mendapatkan hasil

Fulfilled merupakan promise yang berhasil dan menghasilkan nilai

Rejected merupakan promise gagal dan menghasilkan error

Bagian mana yang ditunda oleh `await` dan bagian mana yang tetap dapat merespons pengguna?

`await` menunda kelanjutan function async sampai Promise selesai. Namun, `await` tidak menghentikan seluruh browser atau halaman. Browser tetap dapat merespons interaksi pengguna, menggambar tampilan, dan menjalankan event lain.

Mengapa `fetch` dengan status HTTP 404 tidak otomatis masuk ke `catch`?

Karena `fetch()` menganggap request berhasil secara jaringan meskipun server memberikan status HTTP seperti 404 atau 500. Promise `fetch` tetap dapat fulfilled selama komunikasi dengan server berhasil.

Apa fungsi `response.ok` sebelum `response.json()` dipanggil?

`response.ok` digunakan untuk memeriksa apakah response HTTP berhasil. Nilainya `true` untuk status HTTP yang berhasil, sedangkan `false` untuk status error seperti 404 atau 500.

Apa perbedaan state `empty` dan `error` dari sudut pandang pengguna?

`Empty` berarti permintaan berhasil tetapi tidak ada data yang dapat ditampilkan

`Error` berarti permintaan gagal

Mengapa tombol dinonaktifkan selama proses permintaan data berlangsung?

Agar pengguna tidak dapat menekan tombol berkali-kali ketika request sebelumnya masih berlangsung. Hal ini dapat mencegah request ganda, proses yang tidak diperlukan, atau hasil yang saling bertabrakan.

Mengapa state `loading` harus selalu diakhiri setelah proses selesai, baik berhasil maupun gagal?

Karena `loading` menunjukkan bahwa proses masih berlangsung. Jika `loading` hanya dihentikan ketika berhasil, saat terjadi error tampilan bisa terjebak dalam keadaan `loading`.

Bagaimana Console dan Network saling melengkapi ketika mendiagnosis `fetch`?

Console digunakan untuk melihat pesan error, nilai variabel, dan alur JavaScript. Network digunakan untuk melihat request yang dikirim, URL, status HTTP, response, dan waktu request.

B4. Debugging

Mengapa error pertama pada Console diperiksa sebelum error berikutnya?

Karena error pertama sering menjadi penyebab munculnya error-error berikutnya. Jika error pertama diperbaiki, error yang lain mungkin ikut hilang.

Apa perbedaan gejala, penyebab, dugaan, dan bukti?

Gejala merupakan sesuatu yang terlihat masalah, misalnya tombol yang tidak melakukan apa-apa

Penyebab, hal yang sebenarnya membuat masalah terjadi

Dugaan merupakan perkiraan sementara penyebab masalah

Bukti merupakan hasil pemeriksaan yang mendukung dugaan

Bagaimana typeof membantu diagnosis kesalahan perhitungan?

typeof digunakan untuk mengetahui tipe data dari sebuah variable, jadi dapat mengecek apakah terdapat kesalahan dibagian tipe data

Bagaimana console.count membuktikan handler berjalan lebih dari sekali?

Menghitung berapa kali kode di dalam handler dieksekusi.

Mengapa hanya satu perubahan dilakukan pada setiap percobaan perbaikan?

Agar kita dapat mengetahui perubahan mana yang menyebabkan hasil berubah. Jika banyak bagian kode diubah sekaligus, ketika masalah hilang akan sulit mengetahui perubahan mana yang sebenarnya memperbaikinya.

Apa bukti minimum bahwa perbaikan benar-benar berhasil dan bukan hanya menyembunyikan gejala?

Bukti minimumnya adalah masalah yang sebelumnya terjadi sudah tidak muncul, fungsi yang diharapkan berjalan, dan tidak muncul error baru.

C. Independent Challenge

C1. Kalkulator Harga

Gunakan program pada Console dan function yang memiliki nilai kembalian. Lakukan sedikitnya lima kasus pengujian, termasuk nilai batas Rp100.000, Rp200.000, serta satu input yang tidak valid.

Keputusan	Isian Mahasiswa
Data input dan tipe yang digunakan	
Function yang dibuat dan tanggung jawabnya	
Aturan diskon 10%, 20%, tambahan anggota 5%	
Cara membatasi total diskon maksimal 25%	
Cara menangani harga atau jumlah yang kurang dari atau sama dengan 0	

Kasus Uji	Harga	Jumlah	Status Anggota	Ekspektasi	Aktual	Status
1						☐ Lulus ☐ Gagal

Kasus Uji	Harga	Jumlah	Status Anggota	Ekspektasi	Aktual	Status
2 · batas Rp100.000						☐ Lulus ☐ Gagal
3 · batas Rp200.000						☐ Lulus ☐ Gagal
4						☐ Lulus ☐ Gagal
5 · tidak valid						☐ Lulus ☐ Gagal

Jelaskan kasus nilai batas yang paling penting beserta alasannya:

C2. FAQ Accordion

- ☐ Tiga item FAQ dibuat dengan struktur semantik.
- ☐ Pertanyaan menggunakan button, bukan div yang diberi click handler.
- ☐ Event dipasang dengan addEventListener dan tampilan berubah melalui classList.
- ☐ aria-expanded berubah sesuai keadaan.
- ☐ Hanya satu jawaban terbuka pada satu waktu.
- ☐ Dapat digunakan melalui keyboard tanpa library accordion.

Skenario Uji	Ekspektasi	Hasil Aktual	Bukti
Halaman awal	Semua jawaban tertutup		
Klik FAQ pertama	Jawaban 1 terbuka; aria-expanded=true		
Klik FAQ kedua	Jawaban 2 terbuka; FAQ 1 tertutup		
Klik FAQ aktif lagi	State berubah secara konsisten		
Keyboard	Navigasi menggunakan Tab serta aktivasi menggunakan Enter atau Space berfungsi		

Bagaimana Anda memastikan hanya satu jawaban terbuka pada satu waktu?

C3. Async Data Loader

Aspek Desain	Keputusan Mahasiswa
Sumber data JSON lokal	
Delay buatan selama 500 sampai 1500 ms	
Cara memilih satu item secara acak	

Aspek Desain	Keputusan Mahasiswa
Simulasi kegagalan sebesar 30%	
State loading, empty, error, dan mekanisme percobaan ulang	
Cara mencegah klik berulang agar hasil render tetap konsisten	

Skenario Uji	Ekspektasi	Hasil Aktual	Bukti
Pemuatan normal			
Loading terlihat			
Data kosong			
Simulasi gagal			
Tombol Coba Lagi			
Klik berulang			

Jelaskan alur Promise dan proses asinkron sejak tombol diklik hingga data berhasil ditampilkan atau error ditangani.

D. Tugas Praktikum

D1. Task 1 — JavaScript Logic Builder

Target: menggunakan function murni tanpa DOM untuk melakukan validasi nilai, menentukan kategori dan status, serta membentuk ringkasan data.

Function	Tanggung Jawab	Input	Nilai Kembalian atau Keluaran
validasiNilai	Memeriksa apakah nilai merupakan number yang valid, finite, dan berada pada rentang 0–100.	nilai	true jika valid, false jika tidak valid
tentukanKategori	Menentukan kategori nilai berdasarkan rentang nilai A/B/C/D dan menangani nilai yang tidak valid.	nilai	Kategori "A", "B", "C", "D", atau "Data tidak valid"
tentukanStatus	Menentukan apakah nilai termasuk lulus atau tidak lulus, serta menangani nilai yang tidak valid	Nilai	"Lulus", "Tidak lulus", atau "Data tidak valid"

Function	Tanggung Jawab	Input	Nilai Kembalian atau Keluaran
buatRingkasan	Menggabungkan nama, nilai, kategori, dan status menjadi satu objek ringkasan.	Nama, nilai	Object berisi nama, nilai, kategori, dan status.

Kasus	Input Nilai	Kategori Diharapkan	Status Diharapkan	Hasil Aktual	Status
1	0	D	Tidak lulus	Tidak lulus	✓
2	59	D	Tidak lulus	Tidak lulus	✓
3	60	C	Lulus	Lulus	✓
4	69	C	Lulus	Lulus	✓
5	70	B	Lulus	Lulus	✓
6	84	B	Lulus	Lulus	✓
7	85	A	Lulus	Lulus	✓
8	101	null	Data tidak valid	Data tidak valid	✓
9	"80"	null	Data tidak valid	Data tidak valid	✓

Mengapa 59, 60, 69, 70, 84, dan 85 merupakan boundary value penting?

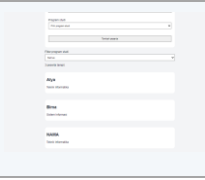
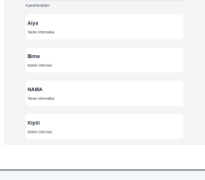
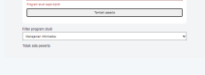
Nilai 59, 60, 69, 70, 84, dan 85 merupakan *boundary value* penting karena nilai-nilai tersebut berada di sekitar batas perubahan kategori nilai. Nilai 59 dan 60 menguji batas kelulusan, nilai 69 dan 70 menguji batas perubahan kategori dari C ke B, sedangkan nilai 84 dan 85 menguji batas perubahan kategori dari B ke A. Dengan menguji nilai sebelum dan tepat pada batas tersebut, dapat diketahui apakah program memberikan kategori dan status yang sesuai dengan aturan yang telah ditentukan.

Tuliskan perubahan nilai batas yang diuji, prediksi dampaknya, dan hasil pengujian yang diperoleh.

Perubahan nilai batas yang diuji adalah 59 ke 60, 69 ke 70, dan 84 ke 85. Pada perubahan 59 ke 60 diprediksi status berubah dari Tidak lulus menjadi Lulus dan kategori berubah dari D menjadi C. Hasil pengujian menunjukkan bahwa 59 menghasilkan kategori D dan Tidak lulus, sedangkan 60 menghasilkan kategori C dan Lulus. Pada perubahan 69 ke 70 diprediksi kategori berubah dari C menjadi B, dan hasil pengujian menunjukkan 69 menghasilkan C sedangkan 70 menghasilkan B. Pada perubahan 84 ke 85 diprediksi kategori berubah dari B menjadi A, dan hasil pengujian menunjukkan 84 menghasilkan B sedangkan 85 menghasilkan A. Hasil tersebut sesuai dengan aturan kategori yang dibuat dalam program.

D2. Task 2 — DOM dan Form Controller

Skenario	Tindakan	Hasil yang Diharapkan	Hasil Aktual	Bukti
----------	----------	-----------------------	--------------	-------

Skenario	Tindakan	Hasil yang Diharapkan	Hasil Aktual	Bukti
Awal	Muat halaman	Dua kartu peserta tampil	Dua kartu peserta tampil sesuai data awal	
Kosong	Submit tanpa isi	Dua pesan error; data tidak bertambah	Dua pesan error muncul dan tidak ada kartu baru	
Nama pendek	Nama 2 karakter	Error hanya pada nama	Pesan error hanya muncul pada field nama	
Valid	Isi semua field	Satu kartu baru ditampilkan	Satu kartu peserta baru berhasil ditampilkan	
Submit kedua	Tambah peserta lain	Data lama tidak terduplikasi	Peserta baru bertambah dan data lama tetap hanya muncul satu kali	
Filter ada	Pilih satu prodi	Hanya program studi tersebut yang ditampilkan	Hanya peserta dari program studi yang dipilih ditampilkan	
Filter kosong	Pilih Manajemen Informatika	Pesan "Tidak ada peserta" ditampilkan	Pesan "Tidak ada peserta" tampil dan tidak ada kartu peserta	
Keyboard	Tab lalu Enter	Seluruh kontrol dapat digunakan melalui keyboard	Kontrol dapat dipindahkan dengan Tab dan dijalankan dengan Enter	

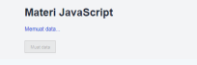
Keputusan Teknis	Penjelasan / Bukti
Cara menggunakan createElement dan textContent	createElement digunakan untuk membuat elemen HTML melalui JavaScript, seperti article, h2, dan p. Setelah elemen dibuat, textContent digunakan untuk memasukkan nama dan program studi. Cara ini membuat data peserta dapat ditampilkan secara dinamis tanpa menulis HTML satu per satu.

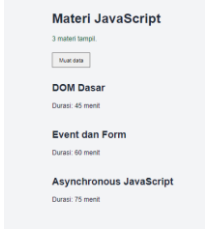
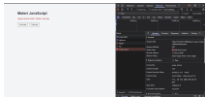
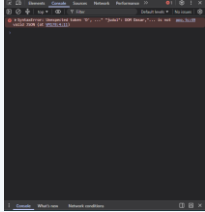
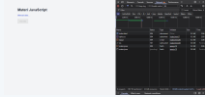
Keputusan Teknis	Penjelasan / Bukti
Cara replaceChildren mencegah duplikasi	replaceChildren() digunakan untuk menghapus seluruh isi elemen daftar sebelum kartu peserta ditampilkan kembali. Hal ini penting karena renderPeserta() dapat dipanggil berkali-kali, misalnya saat menambah peserta atau mengganti filter. Dengan menghapus isi sebelumnya, kartu tidak akan muncul secara berulang.
Cara validasi dipisahkan dari render DOM	Validasi dibuat dalam fungsi validasiPeserta() agar tugas pengecekan data terpisah dari tugas menampilkan data. Fungsi tersebut hanya memeriksa nama dan program studi, kemudian mengembalikan hasil berupa valid, errorNama, dan errorProdi. Pemisahan ini membuat kode lebih mudah dipahami dan diperbaiki.
Cara aria-invalid diperbarui	aria-invalid digunakan untuk memberikan informasi kepada teknologi bantu bahwa input memiliki kesalahan. Jika nama atau prodi tidak valid, nilainya diatur menjadi true. Jika input sudah benar, nilainya menjadi false. Pada kode digunakan setAttribute() untuk memperbarui nilai tersebut.
Cara filter menjaga array peserta asli	Filter menggunakan method filter() untuk membuat array baru berdasarkan program studi yang dipilih. Array peserta tidak diubah atau dihapus. Karena itu, ketika pengguna memilih semua, seluruh data peserta dapat ditampilkan kembali.

Kesalahan program yang paling penting yang ditemukan pada Task 2 serta cara memperbaikinya:

validasi input ketika nama peserta kurang dari 3 karakter atau program studi belum dipilih. Jika input tidak memenuhi aturan, program harus menampilkan pesan kesalahan dan tidak menambahkan peserta ke dalam daftar. Cara memperbaikinya adalah dengan menggunakan fungsi validasiPeserta() untuk memeriksa nama dan program studi, kemudian mengatur aria-invalid serta menampilkan pesan pada elemen error. Jika hasil validasi tidak valid, proses dihentikan dengan return. Setelah input valid, data peserta baru ditambahkan menggunakan push(), form di-reset, pesan error dikosongkan, dan daftar peserta ditampilkan kembali menggunakan renderPeserta(). Dengan perbaikan tersebut, program dapat mencegah data yang tidak sesuai masuk ke dalam daftar peserta.

D3. Task 3 — Asynchronous JavaScript Data Loader

Skenario	Cara Membuat	State dan Hasil yang Diharapkan	Hasil Aktual	Bukti
Idle	Muat halaman	Belum ada permintaan; tombol aktif	Halaman menampilkan teks “Tekan tombol untuk memuat data.” dan tombol Muat data dalam keadaan aktif. Belum ada kartu materi yang ditampilkan.	
Loading	Klik Muat data	Pesan loading; tombol dinonaktifkan	Setelah tombol Muat data diklik, status berubah menjadi “Memuat data...”, daftar materi dikosongkan, dan tombol Muat data dinonaktifkan selama proses fetch.	

Skenario	Cara Membuat	State dan Hasil yang Diharapkan	Hasil Aktual	Bukti
Success	JSON valid	Tiga kartu tampil; status 200	Dengan data/materi.json yang valid, terdapat 3 kartu materi yang tampil: DOM Dasar, Event dan Form, serta Asynchronous JavaScript. Status berubah menjadi "3 materi tampil." dan request berhasil dengan HTTP 200.	
Empty	JSON = []	Pesan data kosong; bukan error	Jika isi JSON diubah menjadi [], tidak ada kartu yang ditampilkan dan status berubah menjadi "Tidak ada materi.". Kondisi ini tidak dianggap sebagai error.	
Path salah	Ubah URL fetch	State error dan status 404 terlihat pada Network	Jika URL fetch diubah ke file yang tidak ada, request menghasilkan HTTP 404. Error ditangkap oleh catch, kemudian status berubah menjadi "Gagal memuat materi. Silakan coba lagi." dan tombol Coba lagi muncul.	
JSON rusak	Rusak sintaks JSON	Parsing gagal dan state berubah menjadi error	Jika sintaks JSON tidak valid, proses response.json() gagal melakukan parsing. Error masuk ke catch, Console menampilkan error parsing, dan antarmuka menampilkan "Gagal memuat materi. Silakan coba lagi.".	
Server mati	Hentikan local server	Fetch ditolak; antarmuka menampilkan error	Jika local server dihentikan, status berubah menjadi "Gagal memuat materi. Silakan coba lagi." dan tombol Coba lagi muncul.	
Klik ganda	Klik tombol dua kali dengan cepat	Satu proses; daftar tidak ganda	Ketika tombol Muat data diklik dua kali dengan cepat, tombol langsung menjadi disabled sehingga klik berikutnya tidak menjalankan proses baru. Data tetap tampil satu kali dan tidak terjadi daftar kartu ganda.	

Pemeriksaan melalui Network dan Console	Temuan
URL permintaan dan status HTTP	URL yang dipanggil adalah data/materi.json. Saat JSON valid, request berhasil dengan status 200.

Pemeriksaan melalui Network dan Console	Temuan
Respons JSON	Respons berisi 3 data materi, yaitu DOM Dasar, Event dan Form, serta Asynchronous JavaScript.
Pesan error akibat path atau status HTTP	Jika path salah, Network menunjukkan 404, kemudian Console menampilkan E halaman menampilkan pesan gagal memuat materi.
Pesan error parsing	Jika JSON rusak, response.json() gagal melakukan parsing dan Console menampilkan SyntaxError/parsing error, kemudian state berubah menjadi error.
Jumlah permintaan ketika tombol diklik dua kali dengan cepat	Hanya 1 proses/request yang dijalankan karena tombol Muat data langsung dinonaktifkan saat proses dimulai.

Jelaskan urutan state yang Anda amati sejak tombol diklik hingga seluruh proses selesai:

Urutan state yang diamati adalah idle → loading → success/error/empty. Pada awal halaman berada pada kondisi idle dengan pesan “Tekan tombol untuk memuat data.” Setelah tombol diklik, state berubah menjadi loading dan tombol dinonaktifkan. Setelah proses fetch selesai, jika data berhasil dimuat dan berisi data maka state menjadi success. Jika JSON kosong maka state menjadi empty, sedangkan jika terjadi kesalahan seperti path salah, JSON rusak, atau server mati maka state menjadi error. Setelah proses selesai, tombol Muat data kembali aktif melalui finally.

Mengapa Coba Lagi sebaiknya memanggil function pemuatan yang sama?

Tombol Coba Lagi sebaiknya memanggil function muatData() yang sama karena seluruh proses pemuatan data, mulai dari mengubah state menjadi loading, mengambil JSON, menangani success/empty/error, hingga mengaktifkan kembali tombol sudah berada di dalam satu function. Dengan begitu, kode tidak perlu ditulis ulang dan perilaku Muat data dan Coba Lagi tetap konsisten.

D4. Homework — Interactive Profile Card

Fitur atau Kondisi	Status	Bukti dan Catatan
Profil dan keterampilan awal dimuat dari data/profile.json	☐ Selesai	
State loading, empty, error, dan Coba Lagi tersedia	☐ Selesai	
Detail toggle dengan classList dan aria-expanded	☐ Selesai	
Tema dapat diganti	☐ Selesai	
Form penambahan keterampilan memvalidasi input kosong	☐ Selesai	
Setiap keterampilan dapat dihapus	☐ Selesai	
Klik berulang tidak menggandakan data	☐ Selesai	
README menjelaskan cara menjalankan local server	☐ Selesai	

Skenario	Ekspektasi	Hasil Aktual	Bukti
Halaman dibuka	Loading lalu profil tampil tanpa error		
Path JSON salah	Pesan error dan Coba Lagi tampil		

Skenario	Ekspektasi	Hasil Aktual	Bukti
JSON kosong	State empty ditampilkan		
Detail diklik dua kali	Terbuka lalu tertutup		
Input kosong	Data tidak ditambah; pesan muncul		
Keterampilan yang valid	Item baru muncul satu kali		
Semua keterampilan dihapus	Pesan untuk kondisi kosong ditampilkan		

Daftar minimal tujuh tangkapan layar pengujian yang disertakan:

No.	Tangkapan layar	Skenario yang Dibuktikan
1		
2		
3		
4		
5		
6		
7		

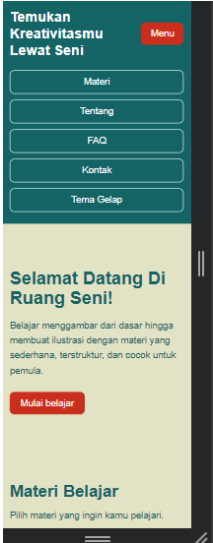
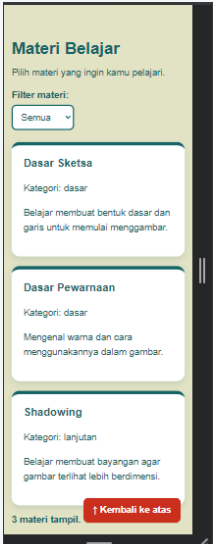
E. Mini Project — Landing Page Interaktif

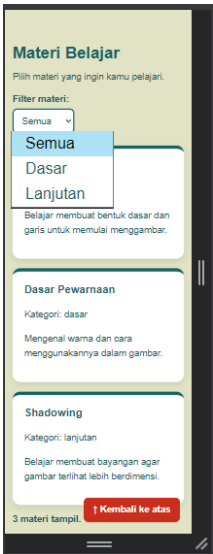
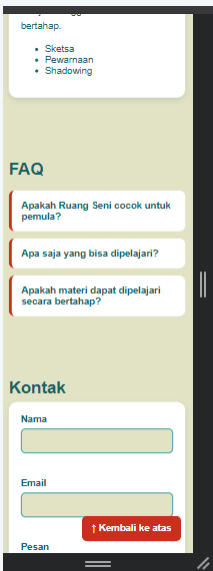
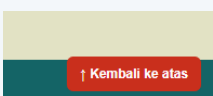
Batas Teknis Mini project menggunakan HTML, CSS, dan vanilla JavaScript. Belum menggunakan fetch, Promise, atau async/await; asynchronous dinilai pada Task 3 dan homework.

E1. Ringkasan Produk

Aspek	Isian Mahasiswa
Nama atau topik landing page	Ruang Seni
Target pengguna	Orang yang sedang belajar gambar
Masalah atau kebutuhan pengguna	Memberikan materi tentang Teknik gambar
URL repository	
URL deployment (jika ada)	

E2. Rencana Fitur dan Acceptance Criteria

Fitur Minimal	Perilaku atau State yang Direncanakan	Acceptance Criteria Mahasiswa	Bukti
Navigasi mobile	Menu navigasi disembunyikan pada layar kecil dan dapat dibuka atau ditutup menggunakan tombol Menu	Saat tombol Menu ditekan, navigasi muncul. Saat ditekan kembali, navigasi tertutup. Navigasi dapat dioperasikan menggunakan keyboard.	
Daftar fitur atau layanan dinamis	Data materi disimpan dalam materi.json	Semua data tampil sebagai kartu materi. Jika data diubah atau ditambah, tampilan mengikuti data tersebut tanpa menulis kartu satu per satu di HTML.	

Fitur Minimal	Perilaku atau State yang Direncanakan	Acceptance Criteria Mahasiswa	Bukti
Filter daftar	Pengguna dapat memilih kategori Semua, Dasar, atau Lanjutan. Daftar materi diperbarui sesuai kategori yang dipilih.	Saat kategori dipilih, hanya materi dengan kategori tersebut yang ditampilkan. Pilihan Semua menampilkan seluruh materi kembali.	
FAQ accordion	Jawaban FAQ disembunyikan secara default dan dapat dibuka atau ditutup dengan menekan pertanyaan. aria-expanded berubah antara true dan false	Saat pertanyaan ditekan, jawaban muncul dan aria-expanded="true". Saat ditekan kembali, jawaban tersembunyi dan aria-expanded="false".	
Form kontak beserta validasi	Form memeriksa nama minimal 3 karakter, email valid, dan pesan minimal 10 karakter. Pesan error ditampilkan jika input tidak valid.	Form tidak dapat dianggap berhasil jika data tidak valid. Setelah semua data valid dan dikirim, muncul pesan "Pesan berhasil dikirim."	
Fitur kembali ke atas atau interaksi lain yang relevan	Tombol Kembali ke atas muncul setelah pengguna melakukan scroll dan dapat digunakan untuk kembali ke bagian atas halaman.	Tombol muncul ketika halaman discroll cukup jauh. Saat tombol ditekan, halaman kembali ke bagian atas dengan efek scroll yang halus.	
Tema atau perubahan visual berbasis class	Tombol Tema Gelap menambahkan atau menghapus class dark pada <body>. Tampilan warna halaman berubah sesuai tema.	Saat tombol ditekan, tema berubah menjadi gelap dan teks tombol menjadi Tema Terang. Saat ditekan kembali, halaman kembali ke tema awal.	

E3. Pemetaan Event, Fungsi Penanganan, dan Perubahan DOM

Fitur	Event	Fungsi Penanganan	Elemen DOM atau Class yang Berubah	State
Navigasi mobile	click pada tombol Menu	bukaMenu()	#main-nav → class is-open, #menu-toggle → aria-expanded	Menu terbuka / tertutup
Filter	change pada #filter-materi	filterDaftarMateri()	#daftar-materi → isi kartu berubah, #status-materi → teks berubah	Semua / Dasar / Lanjutan
FAQ	click pada tombol FAQ	toggleFaq()	.faq-question → aria-expanded, .faq-answer → hidden	Jawaban terbuka / tertutup
Form	submit pada #form-kontak	kirimForm() dan validasiKontak()	#error-nama, #error-email, #error-pesan, #pesan-berhasil, aria-invalid	Valid / Tidak valid / Berhasil
Tema atau state	click pada tombol Tema	ubahTema()	<body> → class dark, #theme-toggle → aria-pressed dan teks tombol	Tema terang / Tema gelap

E4. Riwayat Tahapan

Tahap	Tanggal	Keluaran atau Perubahan	Bukti atau Commit
Audit Modul 1			
Acceptance criteria			
Selector dan data awal			
Fitur 1 sampai 3			
Fitur 4 sampai 7			
Pengujian keyboard dan tampilan responsif			
Perbaikan dan refaktorisasi			
Code selesai			

E5. Acceptance Test Mini Projectf

ID	Kriteria	Status	Bukti dan Catatan
MP-01	Navigasi mobile: button dapat membuka dan menutup menu serta memperbarui aria-expanded secara sesuai.	☐ Lulus	
MP-02	Daftar dinamis: jumlah kartu mengikuti data dan tidak duplikat	☐ Lulus	
MP-03	Filter: subset tepat dan kondisi kosong ditangani	☐ Lulus	
MP-04	FAQ: jawaban terkait terbuka, lainnya tertutup, keyboard bekerja	☐ Lulus	
MP-05	Form: input yang tidak valid ditolak, sedangkan input yang valid menampilkan ringkasan atau pesan keberhasilan.	☐ Lulus	

ID	Kriteria	Status	Bukti dan Catatan
MP-06	Tema atau state: class berubah dan kontras teks tetap terbaca.	☐ Lulus	
MP-07	Tidak ada inline onclick dan tidak memakai innerHTML untuk input pengguna	☐ Lulus	
MP-08	Tidak ada error Console pada alur normal	☐ Lulus	
MP-09	Viewport 320, 768, dan 1024 px tidak merusak tampilan Modul 1	☐ Lulus	
MP-10	Fitur utama dapat digunakan dengan mouse dan keyboard	☐ Lulus	
MP-11	Folder proyek, worksheet, README, dan bukti uji lengkap	☐ Lulus	
MP-12	Saya dapat menjelaskan alur event sampai perubahan DOM	☐ Lulus	

F. Kuesioner Self-Assessment

Skala: 1 belum mampu; 2 mampu dengan banyak bantuan; 3 mampu dengan panduan; 4 mandiri; 5 mampu menjelaskan kepada orang lain.

No.	Pernyataan	1	2	3	4	5
1	Saya dapat menggunakan tipe data, kondisi, dan function.	☐	☐	☐	☐	☐
2	Saya dapat mengolah array of objects.	☐	☐	☐	☐	☐
3	Saya dapat memilih dan mengubah elemen DOM.	☐	☐	☐	☐	☐
4	Saya dapat memasang event listener dan menjelaskan alurnya.	☐	☐	☐	☐	☐
5	Saya dapat memvalidasi form dan menampilkan pesan.	☐	☐	☐	☐	☐
6	Saya dapat menjelaskan sinkron dan asinkron.	☐	☐	☐	☐	☐
7	Saya dapat menggunakan Promise dan async/await.	☐	☐	☐	☐	☐
8	Saya dapat memakai fetch dan memeriksa response.ok.	☐	☐	☐	☐	☐
9	Saya dapat menangani loading, success, empty, dan error.	☐	☐	☐	☐	☐
10	Saya dapat membaca serta memperbaiki error JavaScript.	☐	☐	☐	☐	☐
11	Saya dapat menjelaskan seluruh kode yang dikumpulkan.	☐	☐	☐	☐	☐

Bagian paling menantang pada Modul 2:

Konsep yang sekarang paling siap saya jelaskan kepada orang lain:

G. Peer Assessment

Nama Penilai	Nama Pemilik Mini Project

No.	Kriteria	Skor 1 sampai 5	Catatan Kelebihan dan Saran Perbaikan
1	Kesesuaian fitur: seluruh fitur minimal tersedia		
2	Daftar dinamis: data, render, filter, kondisi kosong bekerja		
3	Interaksi: state konsisten dan dapat diulang		
4	Validasi: input salah ditangani dan pesan membantu		
5	Keyboard: navigasi, FAQ, dan form dapat digunakan tanpa mouse		
6	Stabilitas: tidak ada error atau duplikasi elemen		

Satu umpan balik dari rekan yang akan saya terapkan:

H. Log Bantuan dan Penggunaan AI

Prinsip AI boleh membantu menjelaskan konsep atau memberi hint setelah ada percobaan. Jawaban AI harus diverifikasi melalui dokumentasi, DevTools, Network, atau eksperimen mandiri.

Tanggal atau Fase	Pertanyaan atau Sumber	Intisari Bantuan	Hasil Verifikasi	Penyesuaian atau Perubahan yang Saya Lakukan

Tanggal atau Fase	Pertanyaan atau Sumber	Intisari Bantuan	Hasil Verifikasi	Penyesuaian atau Perubahan yang Saya Lakukan

Jika tidak menggunakan AI, jelaskan strategi debugging dan dokumentasi yang digunakan:

H1. Bukti Pemahaman

Pilih satu saran yang Anda terima. Jelaskan kembali konsepnya dengan bahasa sendiri:

Eksperimen yang membuktikan saran tersebut benar atau tidak:

H2. Batas yang Saya Patuhi

- ☐ Saya menulis prediksi dan percobaan awal sendiri.
- ☐ AI tidak menghasilkan seluruh Task 1, Task 2, Task 3, homework, atau mini project.
- ☐ Pada debugging, saya membaca error, baris, selector, atau nilai aktual sebelum meminta hint.
- ☐ Prompt dan dampaknya dicatat ketika AI digunakan.
- ☐ Saya dapat menjelaskan dan mengubah kode tanpa AI saat coding.

I. Lesson Learnt

Konsep JavaScript: Konsep JavaScript apa yang sekarang dapat Anda jelaskan dengan contoh?

Sinkron dan Asinkron: Apa perbedaan terbesar yang Anda rasakan antara kode sinkron dan asinkron?

Kesalahan yang Memberikan Pembelajaran: Error fetch atau JSON apa yang paling membantu pemahaman?

Keputusan Teknis: Keputusan teknis apa yang Anda ambil dan apa alasannya?

Kemandirian: Bagian mana yang diselesaikan tanpa bantuan?

Umpan Balik Rekan: Umpan balik peer apa yang anda terapkan?

Target Berikutnya: Kompetensi apa yang menjadi target berikutnya?

J. Checklist Akhir Modul 2

J1. Kualitas Kode dan Interaksi

- ☐ File JavaScript dipanggil dengan defer dan tidak menghasilkan error saat halaman dibuka.
- ☐ Nama variabel dan function menjelaskan isi atau tindakan.
- ☐ Input dikonversi dan divalidasi sebelum digunakan.
- ☐ Function logika dapat diuji terpisah dari DOM.
- ☐ Selector sesuai dengan HTML dan tidak ada null yang tidak dijelaskan.
- ☐ Interaksi menggunakan addEventListener, bukan atribut event inline.
- ☐ Perubahan visual memakai class CSS.
- ☐ Button dan form dapat digunakan dengan keyboard.
- ☐ Pesan error spesifik dan tidak hanya mengandalkan warna.
- ☐ Proses asynchronous memiliki state idle, loading, success, empty, dan error.
- ☐ fetch memeriksa response.ok dan menangani kegagalan HTTP atau parsing.
- ☐ Tidak ada pesan error yang belum dijelaskan pada Console.

J2. Kelengkapan Pengumpulan

Komponen	Status	Lokasi atau Nama File
Worksheet PDF	☐	
Task 1 beserta bukti pengujian	☐	
Task 2 beserta tangkapan layar skenario pengujian	☐	
Task 3 beserta bukti dari Console dan Network	☐	
Homework beserta minimal tujuh tangkapan layar pengujian	☐	
Mini project beserta README	☐	
Log bantuan dan penggunaan AI	☐	
Repository atau deployment dapat diakses	☐	

Catatan akhir sebelum mengumpulkan: